

Enterprise AI Platform - 100% Python

Roadmap completo de AI Engineering + desarrollo de software con IA

Objetivo: construir un unico proyecto de portfolio en Python que cubra desde desarrollo asistido por IA hasta LLMs, MCP, RAG, agentes, ML/DL, fine-tuning, multimodal, MLOps/LLMOps, observabilidad, seguridad y produccion.

Capa	Stack recomendado
Backend / APIs	Python + FastAPI + Pydantic
LLMs / Agents	Python + SDKs de modelos + LangGraph / frameworks de agentes
Datos	PostgreSQL + pgvector o Qdrant + Redis
ML / DL	scikit-learn + PyTorch + Hugging Face
MLOps / LLMOps	MLflow + pipelines + evaluaciones + monitoring
Frontend	Opcional: UI Python (Streamlit/Gradio) o frontend separado si luego se desea

TRACK A - Desarrollo de software con IA en Python

Equivale al roadmap .NET original, pero toda la implementacion se realiza en Python.

A1. Desarrollo asistido por IA

Usar asistentes como parte del SDLC y definir reglas para que generen cambios verificables y mantenibles.

Implementar en Python: Claude Code, Copilot, Cursor; AGENTS.md, instrucciones, tests y validaciones.

A2. Skills, Agents & Workflows

Automatizar tareas de desarrollo mediante skills y workflows repetibles.

Implementar en Python: scripts Python, skills, workflows plan -> implement -> test -> verify -> review.

A3. MCP en Python

Crear servidor y cliente MCP, tools propias, recursos y contratos.

Implementar en Python: SDK MCP para Python, FastMCP/implementacion compatible, tools seguras.

A4. Primera feature LLM

Construir chat/feature de IA con streaming, structured outputs y manejo de errores.

Implementar en Python: FastAPI, Pydantic, SDK de LLM, async/streaming, structured outputs.

A5. Embeddings + Semantic Search

Crear embeddings, persistirlos y realizar busqueda semantica.

Implementar en Python: Python, embeddings, cosine similarity, pgvector/Qdrant, metadata filters.

A6. RAG

Implementar ingestion, parsing, chunking, retrieval, grounding y citations.

Implementar en Python: Python, loaders/parsers, embeddings, vector DB, retrieval pipeline.

A7. AI Agents

Construir agentes con tools, estado, memoria, routing y workflows.

Implementar en Python: LangGraph u orquestacion equivalente, tool calling, state, memory, HITL.

A8. Production AI

Endurecer las features para produccion.

Implementar en Python: evals, costos/tokens, guardrails, prompt injection defenses, retries, rate limits, caching.

TRACK B - AI / ML Engineering en Python

Profundiza los fundamentos y la ingeniería de modelos que no cubre un roadmap centrado solamente en aplicaciones LLM.

B1. Machine Learning clasico

Regresión, clasificación y clustering con preprocessing, feature engineering y métricas.

Implementar en Python: pandas, NumPy, scikit-learn.

B2. Deep Learning

Redes neuronales, entrenamiento, backpropagation, optimizadores y fundamentos de Transformers.

Implementar en Python: PyTorch; TensorFlow solo como referencia opcional.

B3. Transformers

Comprender y utilizar modelos Transformer preentrenados.

Implementar en Python: Hugging Face Transformers, tokenizers, inference y evaluación.

B4. Fine-tuning / LoRA

Adaptar un modelo a una tarea/dominio y medir la mejora.

Implementar en Python: PEFT, LoRA/QLoRA, datasets, Hugging Face.

B5. NLP tradicional

Cubrir fundamentos de procesamiento de lenguaje que complementan LLMs.

Implementar en Python: tokenización, TF-IDF, stemming/lemmatization, NER, clasificación.

B6. Multimodal

Procesar texto, PDFs, imágenes y audio dentro de la misma plataforma.

Implementar en Python: modelos multimodales, vision-language, speech-to-text; video opcional.

B7. MLOps / LLMOps

Gestionar experimentos, modelos, prompts, datasets, pipelines y despliegues.

Implementar en Python: MLflow, Model Registry, versionado, CI/CD/CT, monitoring.

B8. Arquitectura AI a gran escala

Diseñar servicios AI para concurrencia, asincronía, escalado y resiliencia.

Implementar en Python: FastAPI services, queues, Redis/cache, workers, autoscaling, cloud.

B9. Evaluación avanzada de RAG/agentes

Medir retrieval, groundedness, calidad de respuesta y task success.

Implementar en Python: golden datasets, retrieval metrics, agent evals, regression suites.

B10. Observabilidad de LLMs

Trazar requests, tools, retrieval, latencia, tokens, costos y errores.

Implementar en Python: OpenTelemetry/observability tooling, metrics, logs, traces, dashboards.

B11. Seguridad avanzada de agentes

Limitar capacidades y proteger datos y herramientas.

Implementar en Python: least privilege, tool permissions, validation, sandboxing, PII, audit logs.

TRACK C - Conceptos avanzados para completar AI Engineering

Estos bloques se integran al proyecto cuando las capacidades base ya funcionan; no necesitan ser proyectos separados.

C1. Prompt Engineering

Disenar prompts reproducibles, estructurados y evaluables.

Implementar en Python: system prompts, few-shot, templates, structured outputs, versionado.

C2. Context Engineering

Controlar que informacion entra al contexto y como se mantiene la memoria.

Implementar en Python: context windows, selection, compression, summarization, short/long-term memory.

C3. Advanced RAG

Mejorar precision y cobertura del retrieval.

Implementar en Python: hybrid search, reranking, query rewriting, multi-query, metadata filtering, GraphRAG opcional.

C4. Advanced Agentic Systems

Pasar de agentes simples a workflows robustos.

Implementar en Python: planning, routing, multi-agent, memory, HITL, durable workflows.

C5. Inference Engineering

Optimizar serving y seleccion de modelos.

Implementar en Python: streaming, batching, caching, quantization, model routing, cost/latency/quality.

C6. AI Data Engineering

Construir pipelines de datos confiables para IA.

Implementar en Python: ingestion, preprocessing, data quality, datasets, lineage, versioning, batch/streaming.

C7. AI Reliability Engineering

Manejar fallos de modelos, APIs y herramientas.

Implementar en Python: timeouts, retries, fallbacks, circuit breakers, idempotency, graceful degradation.

C8. Responsible AI & Governance

Incorporar privacidad, auditoria y gobierno.

Implementar en Python: PII, privacy, explainability, auditability, policies, access control.

Orden recomendado

1. A1-A4: desarrollo AI-first, workflows, MCP y primera feature LLM.
2. A5-A6: embeddings, semantic search y RAG.
3. C1-C3: prompt/context engineering y RAG avanzado.
4. A7: agentes, tools, memoria y HITL.
5. B1-B5: ML, Deep Learning, Transformers, LoRA y NLP.
6. B6: multimodal.
7. B7: MLOps/LLMOps.
8. B9-B11: evaluacion avanzada, observabilidad y seguridad.
9. C4-C8: agentes avanzados, inference, data engineering, reliability y governance.
10. A8 + B8: endurecimiento final y arquitectura de produccion.

Criterio de etapa terminada

- Implementacion funcional y ejecutable.
- Tests o validacion reproducible.
- Integracion con el sistema cuando corresponda.
- Documentacion del problema, arquitectura, decisiones y trade-offs.
- Metricas/evaluaciones para las capacidades de IA.
- No declarar en el portfolio tecnologias que no se hayan implementado realmente.

Resultado final

Un proyecto de portfolio 100% orientado a Python que demuestre AI Engineering end-to-end: APIs de IA, MCP, embeddings, RAG, agentes, ML, Deep Learning, Transformers, fine-tuning, multimodal, MLOps/LLMOps, evaluacion, observabilidad, seguridad, reliability, governance y arquitectura escalable.

Hasta donde llegar: no es necesario profundizar ahora en reinforcement learning avanzado, robotics, CUDA, entrenamiento de foundation models desde cero ni distributed training de gran escala. Esas areas son especializaciones adicionales, no requisitos para un perfil fuerte de AI Engineer aplicado.